L'input con scanf e la grafica ASCII

Lezione 7 — 1IOT, SST-ICT-01 — Acquisire dati dall'utente, backslash come carattere di escape

28/09/2026

1 Ripasso: correzione dell'esercizio del rettangolo

Esempio

Correzione in aula dell'esercizio assegnato la lezione precedente: stampare un **rettangolo** pieno o vuoto di asterischi con un solo `printf`. Un allievo propone la propria soluzione alla lavagna.

Attenzione

Osservazione importante: confrontando le soluzioni di allievi diversi, il numero di asterischi per riga (righe da 9, 6, 5 caratteri a seconda di chi aveva risolto l'esercizio) risultava diverso da persona a persona. Questo non vuol dire che "ognuno può fare come vuole": vuol dire che non era stata data una **specifica precisa** sulle dimensioni del **rettangolo**. Quando una specifica esiste (es. "10 caratteri di larghezza e 7 di altezza"), va rispettata esattamente: un rettangolo di dimensioni diverse, anche se il programma "funziona", non rispetta la **consegna**.

Nota

Due soluzioni diverse per ottenere lo stesso spazio tra gli asterischi laterali: usare più caratteri spazio digitati singolarmente, oppure il carattere di escape `\t` (**tabulazione**). Entrambe sono valide.

2 Input e output: i due strumenti fondamentali

Definizione

La finestra nera che mostra l'esecuzione del programma è la finestra di **output**: quello che l'utente vede e con cui interagisce. Il **codice sorgente** scritto sopra, invece, è quello che tecnicamente l'utente **non vede mai**.

Definizione

Input: le informazioni che noi diamo alla macchina. **Output**: le informazioni che la macchina dà a noi. Il computer usa due dispositivi fondamentali per comunicare con l'utente: lo **schermo** per l'**output** (converte i bit interni in informazioni per noi) e la **tastiera** per l'**input** (nel nostro caso il mouse non viene utilizzato).

Nota

Essere in grado di usare il computer esclusivamente da tastiera, senza **mai** toccare il mouse, è un obiettivo di competenza tecnica per chi lavora con la programmazione.

3 scanf: acquisire un dato da tastiera

Definizione

`scanf` è la **funzione** che permette di acquisire un dato inserito dall'utente durante l'esecuzione del programma e di memorizzarlo in una **variabile**:

```
int raggio;
scanf("%d", &raggio);
```

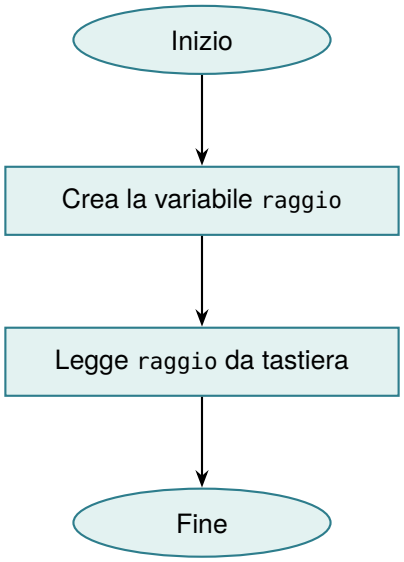


Figura 1: Diagramma di flusso: sequenza di due istruzioni, dichiarazione della variabile e acquisizione del dato da tastiera.

Attenzione

Dettaglio fondamentale: davanti al nome della **variabile**, dentro `scanf`, va inserito il simbolo **`&`** (e commerciale, sopra il tasto 6). Se questo simbolo viene dimenticato, il programma non funziona correttamente. Il motivo verrà spiegato negli anni successivi (è legato al concetto di puntatore): per ora è sufficiente ricordarsi di scriverlo sempre.

Nota

Come per `printf`, lo **specificatore di formato** dentro `scanf` deve corrispondere al tipo della **variabile**: `%d` per un intero, `%f` per un numero **con la virgola** (`float`).

Esempio

Quando il programma incontra un'istruzione **`scanf`**, l'esecuzione si **ferma** e aspetta che l'utente inserisca un valore: solo dopo aver premuto invio, il numero digitato viene memorizzato nella **variabile** e l'esecuzione prosegue.

3.1 Esercizio guidato: il perimetro del cerchio con input da tastiera

Esempio

Codice scritto insieme in aula:

```c
int raggio;
scanf("%d", &raggio);

float perimetro;
perimetro = 2 * 3.14 * raggio;

printf("%d", raggio);
printf("%f", perimetro);
```

Eseguendo il programma con raggio inserito uguale a 7, il risultato stampato è `43.98`, corretto: $2 \times 3{,}14 \times 7 \approx 43{,}96$ (con $\pi = 3{,}14$ approssimato).

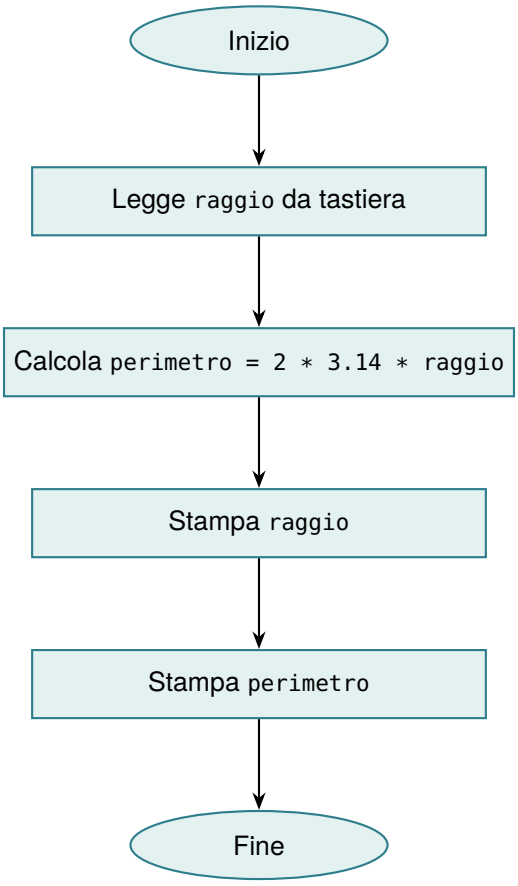


Figura 2: Diagramma di flusso dell'esercizio del perimetro: lettura del raggio, calcolo e stampa dei due valori in sequenza.

Attenzione

Problema di usabilità: così com'è, il programma si limita a mostrare una schermata nera che resta in attesa, senza dire all'utente cosa deve fare. Un programma pensato per essere distribuito ad altri non può funzionare in questo modo: chi lo usa deve sapere cosa inserire, senza dover indovinare.

Definizione

Soluzione: aggiungere un `printf` descrittivo prima dello `scanf`, per guidare l'utente sui dati da inserire, e **contestualizzare** anche l'output finale:

```c
printf("Programma per il calcolo del perimetro di un cerchio\n\n");

int raggio;
printf("Inserisci il raggio del cerchio: ");
scanf("%d", &raggio);

float perimetro;
perimetro = 2 * 3.14 * raggio;

printf("Il perimetro del cerchio vale %0.2f", perimetro);
```

4 Perché usare scanf invece di un valore fisso nel codice

Nota

Se il raggio fosse scritto direttamente nel codice (es. `raggio = 7;`), il programma calcolerebbe sempre e solo il **perimetro** dello stesso cerchio: un programma del genere non è utile a nessuno, esattamente come una calcolatrice che sa fare solo 3×4 e nessun altro calcolo. Dare all'utente la possibilità di scegliere il dato in ingresso è ciò che rende un programma effettivamente utilizzabile.

Definizione

Un altro motivo per usare **`scanf`**: quando si vende un programma, si distribuisce il file **compilato** (eseguibile), non il **codice sorgente**. Se il cliente potesse modificare direttamente il codice sorgente, il programmatore perderebbe il controllo sulla manutenzione e sulle implementazioni future. Dare al cliente la possibilità di cambiare i dati in ingresso, senza dargli accesso al codice, è il modo corretto per rendere un programma flessibile restando proprietari del **codice sorgente**.

Definizione

Alla fine dell'esecuzione, un programma restituisce un **codice di uscita**: lo **zero** è il valore universalmente riconosciuto in informatica per indicare che tutto è andato bene (**`return 0;`**). Un **codice di uscita** diverso da zero segnala che è successo un **errore** durante l'esecuzione (ad esempio, dimenticare la `&` davanti a una **variabile** in `scanf` può generare un **errore** di questo tipo).

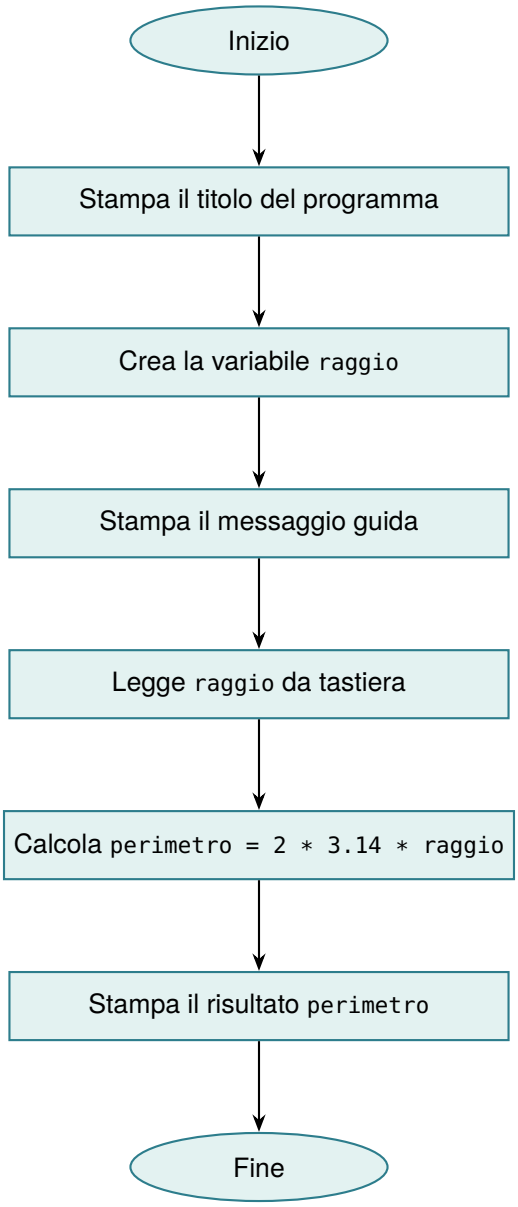


Figura 3: Diagramma di flusso della versione con messaggi guida: titolo, richiesta del dato, lettura, calcolo e stampa del risultato.

5 Grafica ASCII: disegnare con i caratteri di testo

Definizione

La **grafica ASCII** è un tipo di grafica che utilizza esclusivamente caratteri di testo (barre verticali e oblique, virgole, parentesi, trattini, ecc.) per disegnare forme, incluse scritte stilizzate.

Esempio

Sito usato in aula per generare scritte in **grafica ASCII**: `https://patorjk.com/software/taag`. Si inserisce un testo nella casella in alto, si sceglie un **font** (nel nostro caso esclusivamente il font `big`, per uniformità e leggibilità), e il sito genera il testo disegnato con quel font,

pronto per essere copiato.

Esempio

Esercizio assegnato: scrivere un programma in `C` che stampi sulla console il proprio nome e cognome (o cognome e nome) in **grafica ASCII**, usando esclusivamente il font `big` generato dal sito indicato, con uno o più **`printf`**.



Figura 4: Dimostrazione risolta in diretta dal docente: il codice C completo (5 istruzioni `printf`, una per ogni riga della scritta ASCII) e l'output ottenuto, per il nome "Mauro Bortolani" generato dal sito con il font `big` (screenshot reale della schermata condivisa in aula).

6 Il problema del backslash nella grafica ASCII

Attenzione

Il problema riscontrato da quasi tutta la classe: copiando il testo generato dal sito dentro un **`printf`** e stampandolo, il risultato appariva sfasato rispetto all'originale. La causa: il simbolo **backslash** (\) è molto usato nella **grafica ASCII** del font `big` per disegnare le lettere, ma è anche il simbolo che introduce i **caratteri di escape** (**`\n`**, **`\t`**, **`\a`**...).

Definizione

Quando **`printf`** incontra un singolo **`\`** seguito da un carattere, cerca di interpretarlo come un carattere di escape: se la combinazione non corrisponde a un escape valido, il carattere spesso non viene stampato affatto (non produce nulla, o produce un comportamento imprevisto), disallineando tutto il resto del disegno.

Definizione

Soluzione: per stampare un backslash **letterale** con `printf`, va scritto **due volte** di seguito, cioè `\\`. Il doppio **backslash** è il carattere di escape che rappresenta un singolo **backslash** stampato a video:

```
printf("\\ /\n");   // stampa: \ /
```

Ogni singolo carattere `\` presente nel testo copiato dal sito generatore va quindi raddoppiato prima di essere incollato dentro la **stringa** del `printf`.

Nota

Il docente segnala che questo esercizio viene proposto ogni anno e che, quest'anno, ha generato più difficoltà del solito nel gestire il problema in autonomia: un invito a ragionare sul problema (perché il testo non viene stampato correttamente?) prima di cercare scorciatoie.

7 Esercizio: l'area del quadrato con scanf

Esempio

Traccia: scrivere un programma in `C` che chieda all'utente, tramite **`scanf`**, di inserire una misura L in metri (**con la virgola**, quindi `float`), calcoli l'**area** del quadrato di lato L e stampi il risultato specificando cosa rappresenta il numero mostrato.

Esempio

Soluzione realizzata in aula:

```
float L;
printf("Inserisci il lato del quadrato in metri: ");
scanf("%f", &L);

float area;
area = L * L;

printf("L'area del quadrato vale %0.2f", area);
```

Prova con $L = 5{,}2$: il programma stampa `L'area del quadrato vale 27.04` (infatti $5{,}2 \times 5{,}2 = 27{,}04$).

8 Cosa ricordare di questa lezione

- `scanf("%formato", &variabile);` legge un dato da **tastiera** e lo memorizza nella variabile: non dimenticare **mai** la `&`.
- Lo **specificatore di formato** in `scanf` deve corrispondere al tipo della **variabile**, esattamente come in `printf`.
- Un programma con **`scanf`** è più utile e vendibile di uno con i dati fissati nel codice: l'utente

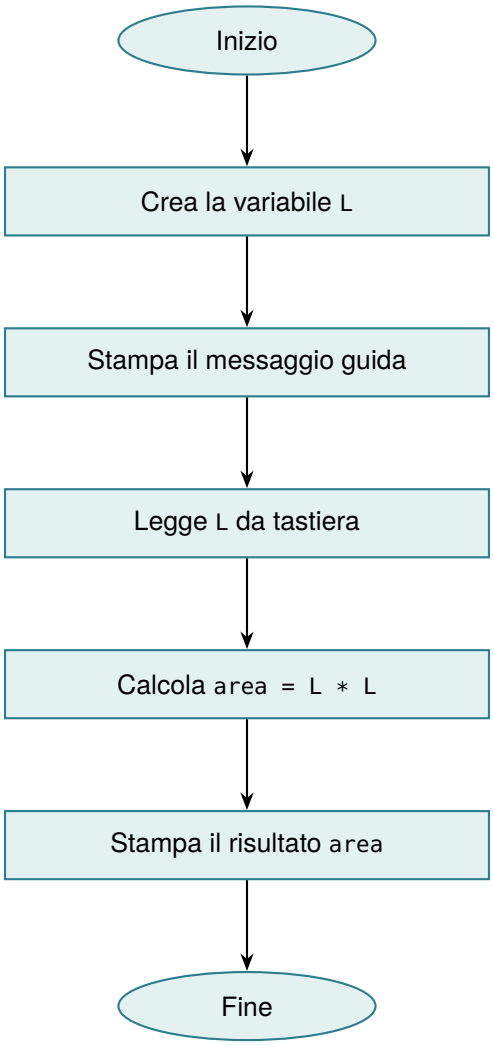


Figura 5: Diagramma di flusso dell'esercizio dell'area del quadrato: dichiarazione, richiesta e lettura del lato, calcolo e stampa dell'area.

può scegliere l'**input** senza avere accesso al **codice sorgente**.

- Prima di ogni `scanf` conviene stampare un messaggio che dica all'utente cosa inserire: un programma senza istruzioni non è utilizzabile da altri.
- Il carattere **backslash** (\) introduce i **caratteri di escape**: per stamparlo letteralmente serve il doppio backslash (\\).
- `return 0;` alla fine del programma segnala che l'esecuzione è terminata correttamente; un codice diverso da zero segnala un **errore**.

Esempio

Da fare per la prossima lezione: completare a casa l'esercizio della **grafica ASCII** con il proprio nome e cognome, gestendo correttamente il doppio **backslash**.